SPHN Hackathon: Secure NL-to-SPARQL over Clinical Knowledge Graphs

1. Introduction to the domain / Motivation for the problem

SIB (SIB Swiss Institute of Bioinformatics) is a Swiss non-profit that builds and operates bioinformatics infrastructure, databases and standards for the life sciences and clinical research community, including the semantic data model behind Switzerland's national health-data network.

The Swiss Personalized Health Network (SPHN) runs a federated framework for sharing routine clinical routine data from hospitals with research institutions such as universities, making it possible to investigate rare diseases and advance personalized health. Hospitals perform semantic coding and standardization locally; patient-level data is never centralized.

Each hospital stores its SPHN-compliant data in its own RDF graph database, reachable only by authorized applications. Together, these decentralized datasets follow the FAIR principles (Findable, Accessible, Interoperable, Reusable) and hold more than 12 billion triples across over 800,000 patients, all of whom gave broad consent.

Patient information is normalized as a knowledge graph using RDF and the wider Semantic Web stack (OWL). SPARQL is the de-facto query language over such graphs: expressive enough to pattern-match anything in them, which is exactly the risk. Even on pseudo-anonymized data, an unrestricted SPARQL endpoint could directly or indirectly re-identify patients, so only specific, pre-approved query shapes should ever reach the backend.

Researchers, on the other hand, rarely have the SPARQL expertise to express their research questions directly as graph queries. Using LLMs to translate natural-language requests into SPARQL has recently shown strong results. This hackathon combines both needs: translating natural language into SPARQL and guaranteeing that the resulting queries are compliant and cannot expose identifiable information about clinical patients.

2. Problem Statement

Given the SPHN schema (version 2026.1), an RDF graph of clinical data modeled with that schema, and a request expressed in natural language, teams must build:

1. A component that translates the natural-language request into a SPARQL query.
2. A component that verifies if the resulting SPARQL query complies with a set of pre-defined security constraints.

Worked example. For the request *"How many patients have a Billed Diagnosis for Type 2 diabetes mellitus (non-insulin-dependent diabetes mellitus)"*, the expected query is:

```
DEFINE sql:select-option "order"
PREFIX sphn: <https://biomedit.ch/rdf/sphn-schema/sphn#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX ucum: <https://biomedit.ch/rdf/sphn-resource/ucum/>
```

```
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
PREFIX einstein: <https://biomedit.ch/rdf/sphn-resource/einstein/shortcuts#>

SELECT (COUNT(?pseudo_id) AS ?countResult)
WHERE {
    {
        SELECT DISTINCT(?g AS ?pseudo_id)
        WHERE {
            GRAPH ?g {
                ?diagnosis0_code rdf:type <https://biomedit.ch/rdf/sphn-resource/icd-
10-gm/E11>
                    OPTION(inference "https://biomedit.ch/rdf/sphn-resource/global")
.
                ?diagnosis sphn:hasCode ?diagnosis0_code .
                ?diagnosis rdf:type sphn:BilledDiagnosis .
            }
        }
    }
}
```

Example compliance constraints the security-check component must enforce:

- Queries about a single, identifiable patient are not allowed.
- Queries must always return aggregated information across all patients (e.g. "give me the number of patients that ...").
- Queries that attempt to match the whole graph without restriction are not allowed.
- (Teams will receive the full, minimal constraint set alongside the dataset — see Section 3.)
-

- An example of a non-compliant request would be: “Return identifiers of patients who have a particular rare disease”.

3. Explanation of the Data

Participants will receive:

- The **SPHN schema, version 2026.1**, publicly available at biomedit.ch/rdf/sphn-schema/sphn.
- The **SHACL** file, **version 2026.1:** the file describes graph patterns any model built according to the SPHN-schema must fulfil.
- Terminologies: additional modelling primitives for modelling clinical data. Please note that the terminologies in the **restricted** section below must not be reshared or reused for any other context that the hackathon:
    - Open source:
        - **GENO**: The Genotype ontology (GENO) provides a graph-based model for the representation of genetic variations described in genotypes implemented in OWL2.
        - **OBI**: The Ontology for Biomedical Investigations (OBI) provides terms with precisely defined meanings to describe all aspects of how investigations in the biological and medical domains are conducted.
        - **SO**: The Sequence Ontology (SO) is a structured controlled vocabulary that aims at facilitating the exchange, analysis and organization of genomic data by

humans and machines alike. The SO unifies the way in which sequence annotations are described.
        - **ORDO**: The Orphanet Rare Disease ontology (ORDO) is derived from the Orphanet database. It captures rare disease information, in particular relationships between diseases, genes and other relevant features to facilitate computational analysis of rare diseases.
    - Restricted:
        - **CHOP**: Swiss medical coding system for procedures performed in hospitals. From version 4 of the RDF files, the 'Z' letter in front of the codes has been removed.
        - **UCUM**: UCUM is a compositional code system intended to include all units of measures being contemporarily used in international science, engineering, and business.
        - **ICD-10**: ICD-10-GM is an international classification for coding of diagnoses. The German Modification is used in Switzerland.
        - **LOINC**: LOINC is is a common language for identifying health measurements, observations, and documents.
        - **SNOMED-CT** terminology. This must not be used or distributed outside of the hackathon. If participants wish to use SNOMED outside the hackathon, they should also register via the MLDS and apply for a license.
- A **zipped dataset of mocked patients** modeled with that schema (will be provided together with the code).
- The **minimal set of security constraints** that any generated SPARQL query must satisfy.
- A **setup script** to spin up a graph database (Virtuoso 7.2.16) pre-loaded with the mocked dataset.

https://mlds.ihtsdotools.org/

4. Additional requirements

- **Open source tooling**: the whole system must consist of tools with open source license including the models used to tackle the challenge (we encourage to try Apertus).
- The system should expose a REST API with one endpoint "/nl-to-sparql". The specification is provided in the code under *API_specification.txt.*A request to that endpoint must return:
    - SPARQL query
    - Number of token used for the query generation and compliance assessment process, in case of multiple instance/agents, the result must include the total number of consumed tokens.

5. Project Evaluation

Main criteria

1. **Efficiency** — total time to produce a compliant SPARQL query (NL translation + compliance check + any evaluation/retry loop).
2. **Compliance correctness** — whether the system's own compliance verdict matches the expected one. Both failure directions are penalized: false positives, i.e. queries the system judges to be compliant but which are actually not, and compliant requests the system wrongly flags as non-compliant.
3. **Query performance** — execution time of the generated query, averaged over 10 runs in isolation.

Minor criteria

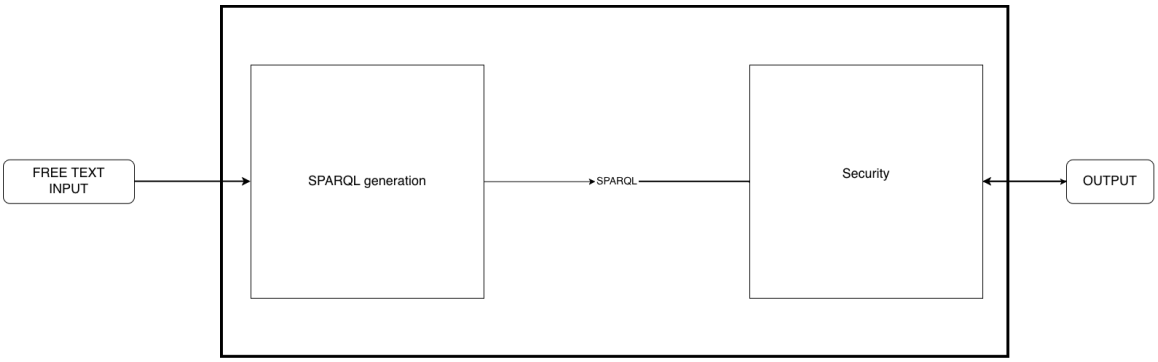
- Resource consumption (e.g. tokens used per request).
- Small models are preferred over large ones.

How judging works. The jury holds a benchmark set of natural-language requests with expected outcomes, each as either compliant or non-compliant:

Query performance is measured afterwards on an independent system, and RAM usage is tracked independently as well.

6. Potential Solution

One possible architecture follows the **evaluator-optimizer** agentic pattern described in Anthropic's Building Effective Agents: a generator produces a candidate SPARQL query, an evaluator checks it against the compliance constraints, and the loop retries or refines the query until it passes (or gives up and reports non-compliance).



Regardless of the internal architecture, teams must implement the following API endpoint:

- `POST /nl-to-sparql` — argument: the natural-language text of the request.

Performance note. The order of triple patterns in a query can materially affect execution time. Virtuoso supports `PRAGMA`-style directives (e.g. to fix join ordering); this can occasionally beats the built-in query optimizer and is worth experimenting with, though it rarely helps.

7. Additional Information

- sib-swiss/sparql-llm — SIB's existing solution for translating natural language into SPARQL; a good reference point (not a mandatory dependency) for the NL-to-SPARQL component.
- SPHN schema, version 2026.1 — the public, canonical schema documentation.
- SPHN - Connector — Tool to convert and anonymize hospital data.
- SPARQL optimization — SPARQL optimization techniques.
- SPARQL- examples – SPARQL query examples for training.

Participants do not need access to any internal SIB or hospital systems: the mocked dataset, schema, constraints and Virtuoso setup script (Section 3) are self-contained and sufficient to build and test a full solution locally.